

Lecture 28: Diffusion II

From maximum likelihood to predicting noise

Derivation follows Ho, Jain & Abbeel (2020). Some panels adapted from Deepia (2025).

Where we stand

Last lecture we built the forward process, and it has **no learned parameters**:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$$

We also found that the obvious training target - predict $\mathbf{x}_{t-1}$ from $\mathbf{x}_t$ - is nearly pure noise and trains badly.

Notation, all of it, from L27. β_t is the noise added at step t ; $\alpha_t = 1 - \beta_t$ is the signal kept at that step; $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$ is the signal kept in total, so $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$. **All three appear today** - the per-step ones show up as soon as we write a single reverse step, and that last identity is what makes the algebra collapse.

Today: start from the one principle we always start from - maximum likelihood - and follow the algebra without shortcuts.

It ends somewhere absurd: a plain mean squared error against the noise we ourselves added.

This is the most mathematical lecture of the chapter. Every step is either something you have seen in L23, or one line of algebra.

Outline

Maximum likelihood, and why it fails

A bound we can actually optimize

The true posterior

Reparameterization: the collapse

From objective to practice

It is twelve lines of code

Where this ends up (so you can stop worrying about it)

Long derivations are easier to follow when you already know the destination. So here it is, before we start:

$$\mathcal{L} = \left\| \varepsilon - \varepsilon_{\theta}(\mathbf{x}_t, t) \right\|^2$$

Add noise to a training image. Ask the network which noise you added. Square the error.

That is the entire training objective. No adversarial game, no sampling from the model during training, no likelihood evaluation.

So why spend a lecture deriving it? Because "predict the noise" is not obviously the right thing to do, and the derivation is what tells us it is - plus *exactly* which approximations we accepted along the way. Two of them come back to bite us in L29.

If you get lost in the middle, come back to this slide. Everything between here and frame 21 is a road from maximum likelihood to that one line.

Maximum likelihood, and why it fails

The right objective, in a form nobody can compute.

The objective we always want

Fit θ so that real images are likely under our model. As in L23, minimize the negative log-likelihood:

$$\min_{\theta} -\log p_{\theta}(\mathbf{x}_0)$$

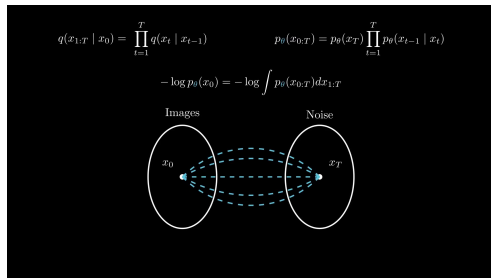
Our model generates by running the whole reverse chain, so p_{θ} is defined over the entire path $\mathbf{x}_{0:T}$:

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t), \quad p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I})$$

We only care about $\mathbf{x}_0$, the finished image. So to get $p_{\theta}(\mathbf{x}_0)$ we must **marginalize away** every intermediate step $\mathbf{x}_1, \dots, \mathbf{x}_T$.

Marginalizing is hopeless

$$p_{\theta}(\mathbf{x}_0) = \int p_{\theta}(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$$



Deepia (2025)

Read the integral literally: to score *one* image, sum over **every possible route** from noise to that image.

There are unimaginably many such paths. The integral is **intractable** - not slow, not approximate, simply not computable.

This is exactly the wall L23 hit with the VAE. And we get past it the same way.

A bound we can actually optimize

If you cannot compute the thing, compute something that hugs it from below.

The same trick as L23: multiply by one

The forward process q is something we know exactly. Insert it, top and bottom:

$$p_{\theta}(\mathbf{x}_0) = \int q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} d\mathbf{x}_{1:T} = \mathbb{E}_q \left[\frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \right]$$

Nothing changed - we multiplied by q/q . But the intractable integral is now an **expectation under a distribution we can sample from**, which is progress.

Recognize this. It is move one of the VAE derivation in L23. The variable names changed; the idea did not.

The bound, derived the way we did it in L23

Take logs and split off a divergence term. Exactly as in L23:

$$\log p_{\theta}(\mathbf{x}_0) = \underbrace{\mathbb{E}_q \left[\log \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right]}_{\text{ELBO}} + \underbrace{D_{\text{KL}}(q(\mathbf{x}_{1:T} | \mathbf{x}_0) \parallel p_{\theta}(\mathbf{x}_{1:T} | \mathbf{x}_0))}_{\geq 0 \text{ always}}$$

A KL divergence is never negative. Drop it and the equality becomes an inequality:

$$\log p_{\theta}(\mathbf{x}_0) \geq \text{ELBO}$$

So maximizing the ELBO pushes up a *floor* underneath the log-likelihood. The gap we threw away is precisely how wrong our reverse process is.

You may see this bound obtained in one line from **Jensen's inequality** instead (that is how the DDPM paper writes it, and how you met Jensen in the probability module).

Same bound, same place. We use the decomposition because it also tells us *what* we gave up.

What the bound looks like once you expand it

Substituting the two Markov factorizations and grinding through Bayes' rule - the DDPM paper does this in its appendix - the ELBO turns into:

$$-\text{ELBO} = \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \| p(\mathbf{x}_T))}_{(1) \text{ no } \theta} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \| p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{(2) \text{ the real work}} - \underbrace{\log p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1)}_{(3) \text{ reconstruction}} \right]$$

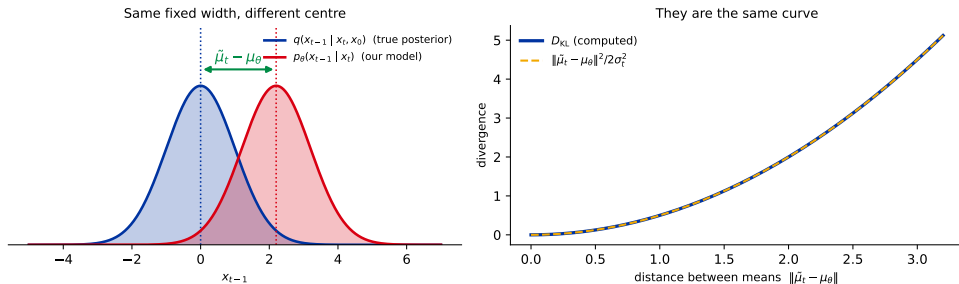
(1) compares "fully noised data" with $\mathcal{N}(\mathbf{0}, \mathbf{I})$. No parameters in it. **Drop.**

(3) the very last denoising step, from $\mathbf{x}_1$ to $\mathbf{x}_0$. Almost no noise is left; contributes little. **Drop.**

(2) a sum of $T - 1$ KL divergences. Everything we care about is here.

The whole training objective is: **make each learned reverse step match some reference reverse step.** Next question - what is that reference?

A reminder of what KL measures



$D_{\text{KL}}(q \parallel p)$ is a measure of how far apart two distributions are: zero exactly when they are identical, larger as they separate. It is not symmetric and it is not a metric, but here it behaves like one - **because we are about to fix both variances to be equal.**

The true posterior

There is an exact answer. It just cheats.

The exact reverse step, in closed form

The reference distribution inside term (2) is $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ - the reverse step *given the clean image*. Unlike the reverse process in general, this one has an exact formula:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\boldsymbol{\beta}}_t \mathbf{I})$$

$$\tilde{\boldsymbol{\mu}}_t = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t, \quad \tilde{\boldsymbol{\beta}}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

Both forward Gaussians are known, so Bayes' rule gives this exactly. It is a Gaussian whose mean is a weighted blend of **where you are** ($\mathbf{x}_t$) and **where you came from** ($\mathbf{x}_0$).

So why do we need a network at all?

Predict first: we just wrote down the exact reverse step. Why not simply use it to generate images?

So why do we need a network at all?

Predict first: we just wrote down the exact reverse step. Why not simply use it to generate images?

Because it needs $\mathbf{x}_0$ - and at generation time $\mathbf{x}_0$ is **exactly what we are trying to produce**. Using it would be circular.

But during **training** we do have $\mathbf{x}_0$: it is the training image. So we can use the true posterior as a **teacher**, and train the network to imitate it from $\mathbf{x}_t$ alone.

Training: teacher sees $\mathbf{x}_0$. Generation: student does not. **That gap is the entire job.**

Choosing the student's form

We make p_θ Gaussian too - the reverse of a Gaussian Markov chain is Gaussian for small steps, and it keeps the algebra finite:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$

And we fix the variance. σ_t^2 is set from the schedule, not learned. So the network's only job is the **mean**.

Concretely, **take** $\sigma_t^2 = \beta_t$ - the simpler of the two choices in Ho et al. (2020). Their other choice is $\tilde{\beta}_t$ from the previous slide; both work, and you need one of them the moment you actually sample.

Two Gaussians, same fixed variance, different means. The KL between them collapses to something very simple.

$$D_{\text{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)) = \frac{1}{2\sigma_t^2} \|\tilde{\mu}_t - \mu_\theta(\mathbf{x}_t, t)\|^2$$

Keep the $\frac{1}{2\sigma_t^2}$ in view - it comes back at the end and we will deliberately throw it away. **The objective is now a squared distance between two means.**

Reparameterization: the collapse

Four lines of algebra turn a variational bound into a mean squared error.

The teacher's mean is in awkward coordinates

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t$$

It depends on $\mathbf{x}_0$, which the student never sees.
Asking the network to match this directly
means asking it to reconstruct $\mathbf{x}_0$.

The idea: rewrite $\tilde{\mu}_t$ in terms of things the student *does* see. It sees $\mathbf{x}_t$. So express everything with $\mathbf{x}_t$ and the noise.

Recall the forward closed form from L27 - the **reparameterization trick**, the same one that made the VAE trainable:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon \quad \implies \quad \mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon \right)$$

Substituting, and watching it simplify

Put that expression for $\mathbf{x}_0$ into $\tilde{\mu}_t$. The $\mathbf{x}_t$ terms combine, and after simplification everything collapses to:

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon \right)$$

Look at what survived. The teacher's mean is $\mathbf{x}_t$ - which the student already has - **minus a multiple of the noise** ε . The only unknown left in the whole expression is ε .

So give the student the same shape, with a network in place of the unknown noise:

$$\mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(\mathbf{x}_t, t) \right)$$

The cancellation

Now subtract the two means. Everything matches except one factor:

$$\tilde{\mu}_t - \mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \cdot \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \left(\varepsilon_{\theta}(\mathbf{x}_t, t) - \varepsilon \right)$$

Square it, put back the $\frac{1}{2\sigma_t^2}$, and the t -th term of the objective is

$$\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \left\| \varepsilon - \varepsilon_{\theta}(\mathbf{x}_t, t) \right\|^2$$

The network's job is to look at a noisy image and say which noise was added. Every $\mathbf{x}_t$ cancelled. What began as an intractable likelihood over all paths is a squared error against a vector we drew ourselves.

From objective to practice

Two more simplifications. One is free, one is not.

Simplification 1: stop summing over t

The objective sums $T - 1$ terms per training example, with T around 1000. Instead, **sample one t uniformly per example**:

$$\mathbb{E}_{t \sim \mathcal{U}\{1, \dots, T\}} [\cdot] \quad \text{estimated with a single draw}$$

This one is free. It is an unbiased Monte-Carlo estimate of the same objective. Over many mini-batches it converges to the full sum, at $1/T$ the cost.

This is why the closed-form $\mathbf{x}_t$ from L27 mattered so much: a random t is useless if reaching it costs t simulation steps.

Simplification 2: throw away the weights

Each term carries the prefactor $\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}$. Ho et al. set it to 1:

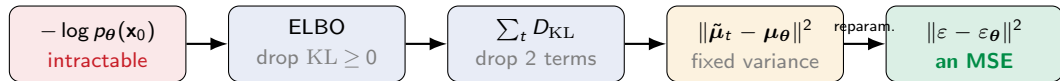
$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \varepsilon} \left\| \varepsilon - \varepsilon_{\theta}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon, t) \right\|^2$$

This one is not free. With the weights gone, $\mathcal{L}_{\text{simple}}$ is **no longer the ELBO**. We are no longer maximizing a likelihood bound.

And it is **better** - for samples. Ho et al. report visibly better images and *worse* likelihood. The reweighting quietly tells the model to care more about the noisy steps, which is where perceptual quality is decided.

We take the trade deliberately. In L29 we come back and ask how anyone measures "better images" in the first place.

The whole road, on one slide



Five steps. Nothing exotic: one algebraic insertion, one non-negativity argument, two terms dropped for good reasons, one change of variables. The reward is an objective a first-year student could implement.

It is twelve lines of code

The payoff for sitting through the algebra.

The training loop, in full

```
# betas: the schedule. abar = cumprod(1 - betas)
for step in range(n_steps):
    x0 = next_batch()
    t = randint(0, T, size=len(x0))
    eps = randn_like(x0)

    a = abar[t]
    xt = sqrt(a)*x0 + sqrt(1 - a)*eps

    loss = mse(eps, model(xt, t))

    loss.backward()
    opt.step(); opt.zero_grad()
```

That is the entire derivation, implemented. Lines 7-8 are the L27 closed form; line 10 is $\mathcal{L}_{\text{simple}}$.

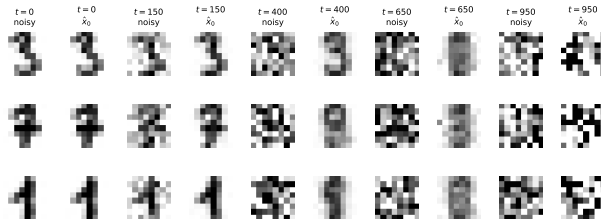
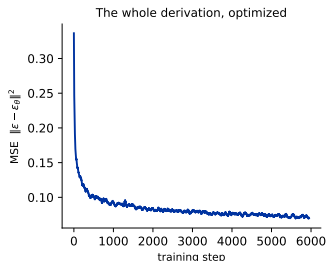
No adversarial game, no discriminator, no balancing act. Just a regression onto noise.

Compare with training a GAN, where the loss going down tells you almost nothing about whether the samples are improving.

And it works

Trained here, on this laptop, on the 8x8 handwritten digits from sklearn:

Trained denoiser: given a noisy digit, predict the clean one (8x8 digits, a deliberately tiny model)



Read this honestly. A deliberately tiny model on tiny images, minutes of CPU. The point is not the quality - it is that at low noise the denoiser recovers the digit almost perfectly, and degrades gracefully as t grows. That degradation is not a bug; it is what the model is supposed to do.

What is ε_θ actually?

We have said "a network" for two lectures. It has to satisfy three constraints:

1. **Image in, image out** at the same resolution - the output ε has the shape of the input.
2. **Conditioned on t** - one network must serve every noise level.
3. Later: **a place to inject text**.

The standard answer is a **UNet**: downsample through convolutions, process, upsample back, with **skip connections** joining matching resolutions so fine detail is not lost in the bottleneck.

You have met every piece:

- ▶ convolutions and transposed convolutions - L16, L19
- ▶ skip connections - L17 (ResNet)
- ▶ encoder-decoder with a bottleneck - L22
- ▶ sinusoidal position encoding, reused here for t - ch9

The digits model above is a miniature of exactly this.

Why "just an MSE" is a big deal

Recall L23b. Training a GAN is a two-player game: the generator improves against a discriminator that is itself moving.

- ▶ no stationary loss to watch - we saw both curves wander and cross
- ▶ **mode collapse** - visible at step 500 of our own run
- ▶ no reliable signal for when to stop; discriminator accuracy never settled

Training a diffusion model is a regression onto a fixed target.

- ▶ the loss is stationary and interpretable
- ▶ more compute reliably helps
- ▶ essentially no instability to manage

This stability, more than any single quality result, is why the field switched.

Recap

1. Maximum likelihood needs an integral over all paths: intractable.
2. Insert q/q , drop a non-negative KL, get the ELBO - the L23 move.
3. Expanded, it is a sum of KLs against the **true posterior**, which is exact but needs $\mathbf{x}_0$.
4. Fix the variance: KL becomes a squared distance of means.
5. Reparameterize; the $\mathbf{x}_t$ terms cancel; what is left is $\|\varepsilon - \varepsilon_\theta\|^2$.
6. Sample one t (free), drop the weights (not free, but better samples).

Next: we can train. We cannot yet *generate* - running the chain backwards has a surprise in it.

L29 shows the sampler, and asks why it insists on adding fresh random noise at every step when its whole job is removing noise.